Kafka Lab Record: Week 9 and Week 10

Topic Management and Kafka Connect using Docker (Confluent Platform 7.4.0)

Week 9: Listing, Modifying and Deleting Topics

Aim

To implement operations such as listing topics, modifying topic configurations, and deleting topics in Apache Kafka.

Software Requirements

- Docker Desktop with Docker Compose
- Git Bash (terminal)
- Images: confluentinc/cp-zookeeper:7.4.0 and confluentinc/cp-kafka:7.4.0

Project Structure

```
Week-9/
├── docker-compose.yml
└── manage_topics.sh
```

Procedure

Step 1: Create the project folder and move into it.

```
mkdir Week-9
cd Week-9
```

Step 2: Create `docker-compose.yml` with ZooKeeper and Kafka services.

```
version: '3.8'

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - 2181:2181

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    depends_on:
      - zookeeper
    ports:
      - 9092:9092
      - 9093:9093
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: 'zookeeper:2181'
```

```
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,PLAINTEXT_HOST://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Step 3: Start the containers and confirm they are running. Note the Kafka container name shown by `docker ps` and use it as `CONTAINER_NAME` in the script (here: kafka-producer-demo-kafka-1).

```
docker-compose up -d
docker ps
```

Step 4: Create a topic named `my-topic` with 3 partitions.

```
docker exec kafka-producer-demo-kafka-1 kafka-topics --create --topic my-topic --
partitions 3 --replication-factor 1 --bootstrap-server localhost:9092
```

Step 5: Create the management script `manage_topics.sh`. Each operation shows the state before and after the change.

```
#!/bin/bash

CONTAINER_NAME="kafka-producer-demo-kafka-1"
BOOTSTRAP_SERVER="localhost:9092"
TOPIC_NAME="my-topic"

run() { docker exec "$CONTAINER_NAME" "$@"; }

case $1 in
  list)
    run kafka-topics --list --bootstrap-server $BOOTSTRAP_SERVER
    ;;
  describe)
    run kafka-topics --describe --topic $TOPIC_NAME --bootstrap-server
$BOOTSTRAP_SERVER
    ;;
  modify)
    echo "=== BEFORE modify ==="
    run kafka-configs --describe --bootstrap-server $BOOTSTRAP_SERVER \
      --entity-type topics --entity-name $TOPIC_NAME
    echo
    echo "=== Applying: retention.ms=3600000 ==="
    run kafka-configs --alter --bootstrap-server $BOOTSTRAP_SERVER \
      --entity-type topics --entity-name $TOPIC_NAME --add-config
retention.ms=3600000
    echo
    echo "=== AFTER modify ==="
    run kafka-configs --describe --bootstrap-server $BOOTSTRAP_SERVER \
      --entity-type topics --entity-name $TOPIC_NAME
    ;;
  delete)
    echo "=== BEFORE delete: topics ==="
    run kafka-topics --list --bootstrap-server $BOOTSTRAP_SERVER
    echo
    echo "=== Deleting topic: $TOPIC_NAME ==="
    run kafka-topics --delete --topic $TOPIC_NAME --bootstrap-server
$BOOTSTRAP_SERVER
```

```
    sleep 3
    echo
    echo "=== AFTER delete: topics ==="
    run kafka-topics --list --bootstrap-server $BOOTSTRAP_SERVER
    ;;
  *)
    echo "Usage: $0 {list|describe|modify|delete}"
    ;;
esac
```

Step 6: Make the script executable.

```
chmod +x manage_topics.sh
```

Step 7: List all topics.

```
./manage_topics.sh list
```

Output:

```
__consumer_offsets
demo-topic
my-topic
test-topic
```

Step 8: Describe the topic.

```
./manage_topics.sh describe
```

Output:

```
Topic: my-topic  TopicId: HXx3_QJ6QmSbMa3RwCxL2A  PartitionCount: 3
ReplicationFactor: 1  Configs:
    Topic: my-topic  Partition: 0  Leader: 1  Replicas: 1  Isr: 1
    Topic: my-topic  Partition: 1  Leader: 1  Replicas: 1  Isr: 1
    Topic: my-topic  Partition: 2  Leader: 1  Replicas: 1  Isr: 1
```

Step 9: Modify the topic configuration (set message retention to 1 hour).

```
./manage_topics.sh modify
```

Output:

```
=== BEFORE modify ===
Dynamic configs for topic my-topic are:

=== Applying: retention.ms=3600000 ===
Completed updating config for topic my-topic.

=== AFTER modify ===
Dynamic configs for topic my-topic are:
  retention.ms=3600000 sensitive=false
synonyms={DYNAMIC_TOPIC_CONFIG:retention.ms=3600000}
```

Step 10: Delete the topic.

```
./manage_topics.sh delete
```

Output:

```
=== BEFORE delete: topics ===
__consumer_offsets
demo-topic
my-topic
test-topic
```

```
=== Deleting topic: my-topic ===

=== AFTER delete: topics ===
__consumer_offsets
demo-topic
test-topic
```

Result

Topics were listed and described, the retention configuration of `my-topic` was changed to 3600000 ms, and the topic was deleted. The topic list before and after the delete confirms the operations.

Week 10: Kafka Connect and Connectors

Aim

To introduce Kafka Connect and demonstrate how connectors integrate Kafka with external systems.

Introduction

Kafka Connect is a framework included with Apache Kafka for moving data between Kafka and external systems without writing custom producer or consumer code.

- Source connector: reads data from an external system and writes it to a Kafka topic.
- Sink connector: reads data from a Kafka topic and writes it to an external system.
- Worker: the process that runs connectors and their tasks. It exposes a REST API (port 8083) to manage connectors.
- Converter: translates data between Kafka's byte format and the connector's format (for example String or JSON).

In this experiment the FileStream connectors are used. The source connector reads lines from `input.txt` into the topic `connect-test`, and the sink connector writes the topic's messages to `output.txt`.

Data flow: input.txt → File Source Connector → topic connect-test → File Sink Connector → output.txt

Software Requirements

- Docker Desktop with Docker Compose
- Git Bash and curl
- Images: cp-zookeeper, cp-kafka and cp-kafka-connect, all version 7.4.0

Project Structure

```
Week-10/
├── docker-compose.yml
├── file-source.json
├── file-sink.json
└── data/
    ├── input.txt
    └── output.txt   (created by the sink connector)
```

Procedure

Step 1: Create the project folders and the input file.

```
mkdir -p Week-10/data
cd Week-10
printf "Hello Kafka Connect\nThis is line 2\nThis is line 3\n" > data/input.txt
cat data/input.txt
```

Step 2: Create `docker-compose.yml`. The `kafka-connect` service is added to the Week 9 setup, and the `data` folder is mounted into the container.

```
version: '3.8'

services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.4.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - 2181:2181

  kafka:
    image: confluentinc/cp-kafka:7.4.0
    depends_on:
      - zookeeper
    ports:
      - 9092:9092
      - 9093:9093
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: 'zookeeper:2181'
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_LISTENERS: PLAINTEXT://0.0.0.0:9092,PLAINTEXT_HOST://0.0.0.0:9093
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1

  kafka-connect:
    image: confluentinc/cp-kafka-connect:7.4.0
    depends_on:
      - kafka
    ports:
      - 8083:8083
    environment:
      CONNECT_BOOTSTRAP_SERVERS: kafka:9092
      CONNECT_REST_ADVERTISED_HOST_NAME: kafka-connect
      CONNECT_REST_PORT: 8083
      CONNECT_GROUP_ID: connect-cluster
      CONNECT_CONFIG_STORAGE_TOPIC: connect-configs
      CONNECT_OFFSET_STORAGE_TOPIC: connect-offsets
      CONNECT_STATUS_STORAGE_TOPIC: connect-status
      CONNECT_CONFIG_STORAGE_REPLICATION_FACTOR: 1
      CONNECT_OFFSET_STORAGE_REPLICATION_FACTOR: 1
```

```
      CONNECT_STATUS_STORAGE_REPLICATION_FACTOR: 1
      CONNECT_KEY_CONVERTER: org.apache.kafka.connect.storage.StringConverter
      CONNECT_VALUE_CONVERTER: org.apache.kafka.connect.storage.StringConverter
      CONNECT_PLUGIN_PATH: /usr/share/java
    volumes:
      - ./data:/data
```

Step 3: Start all services and check that three containers are running.

```
docker-compose up -d
docker ps
```

Step 4: Wait about 30 seconds, then check that the Connect worker is running.

```
curl http://localhost:8083/
```

Output:

```
{"version":"7.4.0-ccs","commit":"...","kafka_cluster_id":"..."}
```

Step 5: Check the connector plugins available in the worker.

```
curl http://localhost:8083/connector-plugins
```

Output (the list contains the FileStream connectors):

```
[{"class":"org.apache.kafka.connect.file.FileStreamSinkConnector","type":"sink","vers
ion":"7.4.0-ccs"},

{"class":"org.apache.kafka.connect.file.FileStreamSourceConnector","type":"source","v
ersion":"7.4.0-ccs"}, ...]
```

Step 6: Create the source connector configuration `file-source.json`.

```
{
  "name": "file-source",
  "config": {
    "connector.class": "org.apache.kafka.connect.file.FileStreamSourceConnector",
    "tasks.max": "1",
    "file": "/data/input.txt",
    "topic": "connect-test"
  }
}
```

Step 7: Register the source connector through the REST API.

```
curl -X POST -H "Content-Type: application/json" --data @file-source.json
http://localhost:8083/connectors
```

Step 8: Check the connector status. Both the connector and its task should be RUNNING.

```
curl http://localhost:8083/connectors/file-source/status
```

Output:

```
{"name":"file-source","connector":{"state":"RUNNING","worker_id":"kafka-
connect:8083"},"tasks":[{"id":0,"state":"RUNNING","worker_id":"kafka-
connect:8083"}],"type":"source"}
```

Step 9: Read the topic with the console consumer to confirm that the file lines reached Kafka.

```
docker exec kafka-producer-demo-kafka-1 kafka-console-consumer --topic connect-test -
-from-beginning --bootstrap-server localhost:9092 --max-messages 3
```

Output:

```
Hello Kafka Connect
This is line 2
This is line 3
Processed a total of 3 messages
```

Step 10: Create the sink connector configuration `file-sink.json`.

```
{
  "name": "file-sink",
  "config": {
    "connector.class": "org.apache.kafka.connect.file.FileStreamSinkConnector",
    "tasks.max": "1",
    "file": "/data/output.txt",
    "topics": "connect-test"
  }
}
```

Step 11: Register the sink connector and view the output file.

```
curl -X POST -H "Content-Type: application/json" --data @file-sink.json
http://localhost:8083/connectors
cat data/output.txt
```

Output:

```
Hello Kafka Connect
This is line 2
This is line 3
```

Step 12: Append a new line to the input file. The source connector picks it up and the sink connector writes it to the output file within a few seconds.

```
echo "This is line 4" >> data/input.txt
sleep 5
cat data/output.txt
```

Output:

```
Hello Kafka Connect
This is line 2
This is line 3
This is line 4
```

Step 13: List the running connectors.

```
curl http://localhost:8083/connectors
```

Output:

```
["file-source","file-sink"]
```

Step 14: Delete both connectors and confirm that none remain.

```
curl -X DELETE http://localhost:8083/connectors/file-source
curl -X DELETE http://localhost:8083/connectors/file-sink
curl http://localhost:8083/connectors
```

Output:

```
[]
```

Result

Kafka Connect was set up with Docker, and two connectors were created through the REST API. The file source connector moved data from `input.txt` into the Kafka topic `connect-test`. The file sink connector moved the same data from the topic to `output.txt`. A line added later flowed through the pipeline automatically, which shows how connectors integrate Kafka with external systems without custom code.

Conclusion

Week 9 covered the basic topic management operations: list, describe, modify configuration and delete. Week 10 introduced Kafka Connect and showed a complete source-to-sink data pipeline using connectors.